Neo-AA：跨生态系统账户抽象与异构签名验证

R3E Network

面向 Neo 生态系统

摘要

本文介绍 Neo-AA——一种 Neo N3 账户抽象架构，能够在一个共享主合约中统一原生 Neo 见证授权与 EVM EIP-712 类型化签名。核心设计挑战在于异构性：Neo N3 通过基于 secp256r1 的见证进行交易授权，而以太坊风格的工作流则通过对类型化数据进行 secp256k1 签名恢复来完成授权。Neo-AA 无需为每个用户部署独立的钱包合约，而是将每个逻辑账户标识符绑定到一个确定性抽象账户地址（作为面向用户的身份），而逻辑 `accountId` 则作为内部存储、索引和验证的协调键。

系统贡献四项核心内容：其一，引入混合授权协议，将 Neo 见证与恢复的 EVM 签名者地址合并计入一个阈值策略；其二，通过验证代理脚本提供确定性地址生成，无需为每个账户部署合约；其三，通过 `.matrix` 域名解析与基于地址的账户查询实现人类可读的账户发现；其四，提供可插拔的恢复架构，包括 Argent 风格的监护人集合、Safe 风格的时间锁恢复、Loopring 风格的监护人哈希恢复，以及由预言机控制的 Dome 不活跃恢复流程。本文全面描述系统架构、状态模型、工作流程和安全属性，并与当前实现及测试网验证流程保持一致。

目录

1 引言 **3**
- 1.1 动机 . . . 3
- 1.2 Neo N3 的挑战 . . . 3
- 1.3 挑战 . . . 3
- 1.4 解决方案：混合授权协议 . . . 3
- 1.5 主要贡献 . . . 4
- 1.6 .matrix 域名集成 . . . 4
- 1.7 恢复验证器架构 . . . 5
- 1.8 Dome 不活跃恢复协议 . . . 6

2 账户模型与生命周期 **6**
- 2.1 账户创建与结构 . . . 6
- 2.2 确定性地址生成 . . . 7
- 2.3 交易结构与验证入口 . . . 8
- 2.4 混合授权交易结构 . . . 8

3 混合授权协议 **8**
- 3.1 协议概述 . . . 8
- 3.2 核心验证方法 . . . 9
  - 3.2.1 Verify 方法：两阶段验证模型 . . . 9

3.2.2 CheckNativeSignatures：纯 Neo 签名验证 . . . 9
3.3 CheckMixedSignatures：统一阈值算法 . . . 9
3.4 EIP-712 签名验证详情 . . . 11
3.5 状态管理与随机数机制 . . . 11
3.6 EIP-712 类型化数据结构 . . . 11
3.7 验证算法 . . . 12

4 安全分析 **12**
4.1 威胁模型 . . . 12
4.2 形式安全定义 . . . 13

5 相关工作 **13**
5.1 账户抽象系统 . . . 13
5.2 阈值授权与跨生态系统授权 . . . 14
5.3 Neo-AA 的定位 . . . 14

6 讨论 **14**
6.1 局限性 . . . 14
6.2 设计权衡 . . . 14
6.3 运营架构 . . . 15
6.4 未来方向 . . . 15
6.5 部署详情 . . . 15

7 结论 **16**

1 引言

1.1 动机

随着区块链生态系统的成熟，用户越来越需要跨多个网络管理数字资产。Neo N3 生态系统拥有丰富的原生功能和成熟的工具链，但对来自以太坊生态背景的用户而言设置门槛较高。Neo N3 的验证机制与以太坊截然不同：Neo N3 采用基于 secp256r1（P-256）曲线的见证，通过 `Runtime.CheckWitness` 进行授权；而以太坊则采用基于 secp256k1 的 ECDSA 签名（通常通过 `ecrecover` 进行链上恢复）。

1.2 Neo N3 的挑战

Neo N3 缺乏对 EVM 兼容钱包原生的支持，由此造成了以下具体障碍：

- EVM 用户无法直接使用 MetaMask 等钱包与 Neo N3 交互，需要专用钱包软件；
- 在 Neo N3 上的多签或阈值授权通常需要合约级别的复杂协调逻辑；
- 账户管理原语缺乏统一标准，每种 dApp 各行其是；
- 在发现与安全恢复方面，目前尚无规范路径。

1.3 挑战

构建跨生态账户抽象系统时，需克服以下核心挑战：

核心技术挑战

签名异构性

Neo 使用 secp256r1，而 EVM 使用 secp256k1。两者不可直接互换，必须在同一授权框架内共存。

部署开销

传统智能合约钱包（如 Safe）为每个用户部署独立合约，成本高昂且难以扩展。

重放保护

EVM 签名必须绑定到特定操作以防止重放攻击，同时还须兼容 Neo 的原生见证机制。

可发现性

用户需要一种人类可读的方式来发现和引用账户，而不必记忆复杂的哈希标识符。

1.4 解决方案：混合授权协议

Neo-AA 引入了一种混合授权协议：单个 Neo N3 主合约同时接受原生 Neo 见证和 EVM EIP-712 类型化签名，并将两者合并计入同一阈值。核心架构观察在于：两种签名者类型最终都可归一化为地址——Neo 见证通过 `Runtime.CheckWitness` 进行验证，而 EVM 签名则在链上完成验证，映射为恢复出的以太坊风格地址，并与相同存储的角色集进行比对。

Neo-AA 设计原则

确定性寻址

系统为每个账户派生一个确定性验证代理脚本，转发验证请求至主合约。账户在外部以确定性地址进行标识，`accountId` 仅作内部协调键。

地址优先路由

公共执行路径同时暴露逻辑标识符入口点和基于地址的便捷封装。用户界面只使用确定性 AA 地址或绑定的 `.matrix` 域名，而非不透明的逻辑标识符。

EIP-712 集成

EVM 签名者通过签署与 AA 地址、目标合约、方法哈希、参数哈希、随机数及截止时间相绑定的类型化数据来授权操作。主合约在链上重建摘要，恢复签名者地址，并将其送入与 Neo 执行相同的权限引擎。

1.5 主要贡献

核心贡献

混合授权协议

一种统一阈值机制，将 Neo 见证与 EVM EIP-712 签名合并计入单一授权阈值，无需桥接合约或同步副本状态即可实现跨生态账户抽象。

确定性地址生成

零部署账户模型：验证脚本确定性生成并路由至共享主合约，消除每账户部署成本的同时保证地址稳定性。

统一阈值算法

`CheckMixedSignatures` 算法将 Neo 的 `Runtime.CheckWitness` 与 EVM 签名恢复无缝集成，通过单一验证路径支持纯 Neo、纯 EVM 及混合授权三种场景。

形式安全分析

严格的安全定义和证明，在两种签名方案的标准密码学假设下，验证了阈值安全性、重放保护和授权正确性。

1.6 .matrix 域名集成

Neo-AA 将 Matrix 命名系统作为发现层集成，而非授权根。当用户输入 `alice.matrix` 时，前端从 Matrix 合约解析域名所有者，向 AA 合约查询与该所有者关联的已绑定地址，并展示发现的 AA 账户供选择。

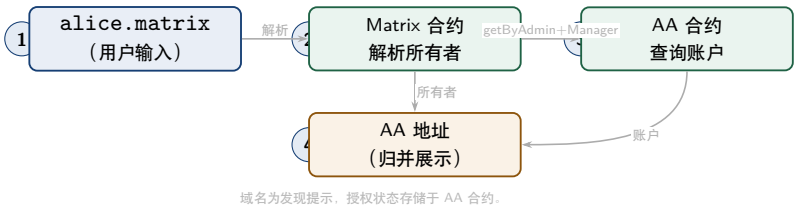

图 1: `.matrix` 域名解析流程：域名解析为所有者地址，再查询 AA 合约获取绑定账户。

1.7 恢复验证器架构

Neo-AA 通过标准化接口支持可插拔的恢复验证器。每个账户可选择设置一个实现如下接口的恢复验证器合约：

```
interface IRecoveryVerifier {
  bool verifyRecovery(
    byte[] accountId,
    byte[] operation,
    byte[] proof
  );
}
```

Argent 风格的监护人恢复

链上维护的状态

- **监护人集合**：监护人地址的序列化数组（`byte[]`），同时支持 Neo 和 EVM 地址。
- **监护人阈值**：所需的最小监护人签名数量（m of n）。
- **待处理操作**：恢复操作哈希到待处理恢复请求的映射。
- **时间锁周期**：发起和执行之间可配置的延迟（默认 24 小时）。

恢复操作生命周期

阶段一：发起

监护人提交 `accountId`、替换管理员和拟定阈值，合约计算哈希 $h = H(\text{accountId}\|\text{newAdmins}\|\text{newThreshold}\|\text{nonce})$，存储待处理操作并发出 `RecoveryInitiated` 事件。

阶段二：监护人批准

其他监护人提交操作哈希及其批准签名，验证器验证签名并递增批准计数。

阶段三：执行

阈值满足且时间锁到期后，任何人均可调用 `executeRecovery(operationHash)` 执行恢复。

取消 账户当前所有者可随时通过成功执行任意授权交易来取消任何待处理的恢复操作。

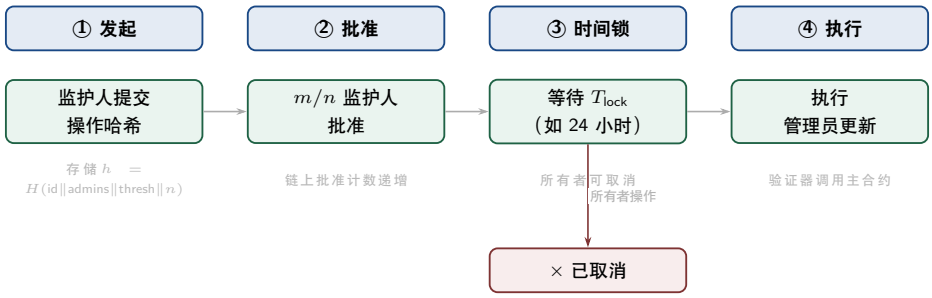

图 2: 监护人恢复生命周期（Argent 风格）：四个阶段依次为发起、批准累积、时间锁取消窗口及执行更新。

1.8 Dome 不活跃恢复协议

Dome 机制为长期不活跃的账户提供最后手段恢复。系统跟踪每个账户的最后活跃时间戳，当 $\text{Runtime.Time} - \text{lastActivity} > T_{\text{inactivity}}$ 时，账户被标记为不活跃（默认周期为 180 天）。

Dome 不活跃恢复流程

不活跃检测

系统持续追踪账户活跃度；超过阈值则允许触发恢复流程。

预言机控制发起

指定恢复地址可发起 Dome 恢复，但执行需要外部预言机批准以确认真实不活跃状态。

预言机验证与恢复

授权预言机在链下验证后签署批准。批准后需等待最终时间锁（默认 7 天）。在此窗口内，原所有者可通过任意授权交易单方面取消恢复。

2 账户模型与生命周期

2.1 账户创建与结构

Neo-AA 中的抽象账户是一个共享合约钱包实例，具有两个标识符：逻辑 `accountId` 和确定性 AA 地址。`accountId` 是内部存储键和验证参数；确定性 AA 地址是面向用户的公开身份。这种分离对实现和用户体验都至关重要：用户通过地址对 AA 进行资金转入、发现和调用，而合约则使用逻辑标识符进行紧凑索引、地址派生，以及兼容内部验证例程。

身份模型

逻辑身份： `accountId` 创建一次，验证后作为主合约内部持久存储的规范键。

操作身份： 确定性 AA 地址是钱包、中继器和下游合约查询和交互的签名者身份。

绑定层： 主合约维护 `accountId` ↔ `accountAddress` 的权威双向映射。

用户规则： 创建后，所有操作路径强烈优先使用 AA 地址；`accountId` 仅保留用于初始化、存储索引和基础验证器兼容。

账户创建流程

1. 客户端选择 `accountId`，并确定性地派生对应的 AA 验证代理地址。
2. 通用创建路径调用地址绑定创建封装方法 `createAccountWithAddress(...)`。
3. 主合约验证逻辑标识符是新的、提供的地址与确定性代理派生匹配，且角色阈值格式正确。
4. 合约将管理员和管理者集合存储在 `accountId` 的规范存储键下，初始化活跃度追踪，并绑定双向地址映射。
5. 交易发起者若不在管理员集合中，则自动被插入，确保启动创建不会造成孤立账户。

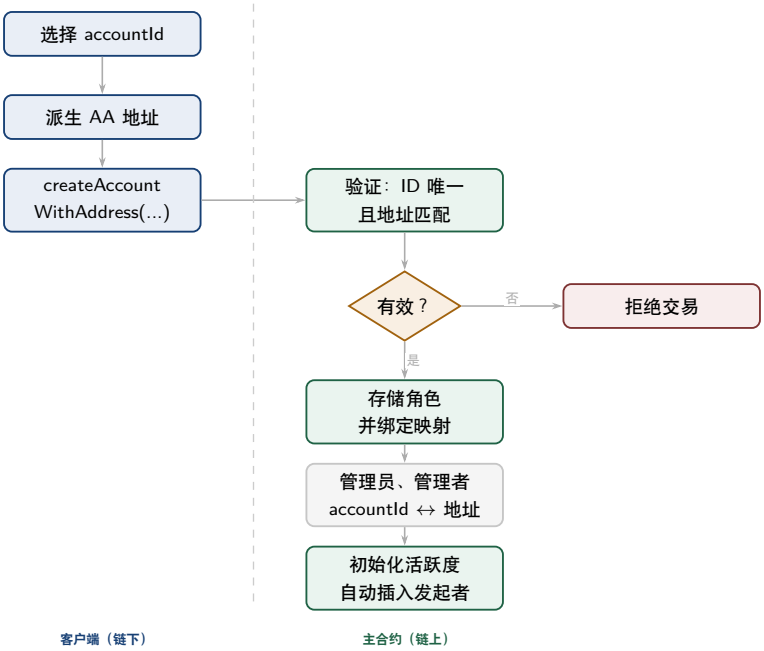

图 3: 账户创建与绑定生命周期：客户端从 `accountId` 派生确定性地址，主合约验证唯一性后存储角色集并绑定双向地址映射。

2.2 确定性地址生成

每个抽象账户均有一个由 `accountId` 和主合约哈希确定性派生的 Neo 地址。该确定性地址是账户的公开身份、签名者句柄和发现键。

确定性代理寻址

Neo 地址计算如下：

$$\text{address} = \text{Base58Check}(\text{version} \| \text{Hash160}(\text{verifyScript}))$$

其中 Neo N3 的 `version = 0x35`。脚本哈希从代理字节码派生，并在 Base58Check 编码前反转为 Neo 的 `UInt160` 表示。

核心属性

确定性构造

相同的 `accountId` 始终产生完全相同的验证代理地址。

零部署成本

无需为每个账户发送部署交易。

地址优先可用性

账户派生后即可立即通过地址进行充值、发现和引用。

绑定完整性

主合约拒绝将 `accountId` 绑定到其数学派生代理地址以外的任何地址。

2.3 交易结构与验证入口

交易完整性规则

所有使用抽象账户的交易必须将确定性验证地址作为签名者。这将严格触发主合约中的验证入口点。

标准纯 Neo 交易结构

1. **脚本**：调用基于地址的封装方法 `executeByAddress(accountAddress, targetContract, method, args)`。
2. **签名者**：包含确定性验证地址（脚本哈希）的数组。
3. **见证**：包含空调用脚本（或签名数据）和确定性验证脚本的见证对象数组。

2.4 混合授权交易结构

混合交易结构

1. **脚本**：调用基于地址的元交易封装方法 `executeMetaTxByAddress(...)`。
2. **签名者**：确定性验证地址（与纯 Neo 交易相同）。
3. **属性**：包含 EVM 签名（r, s, v）、随机数和截止时间的自定义交易属性。
4. **见证**：Neo 见证（如果仅使用 EVM 授权，则可为空）。

3 混合授权协议

3.1 协议概述

混合授权协议阶段

阶段一：EVM 签名生成

（链下）EVM 签名者构造 EIP-712 类型化数据并使用以太坊钱包签名。签名为 65 字节的 (r, s, v)，其中 $r, s \in \mathbb{F}_p$ 为 ECDSA 签名分量，$v \in \{27, 28\}$ 为恢复标识符。

阶段二：交易构造

构建 Neo 交易，包含：脚本（对主合约封装方法的调用）、签名者（账户确定性验证地址）、属性（作为自定义属性嵌入的 EVM 签名）、见证（Neo 签名者占位符）。

阶段三：Neo 见证收集

Neo 签名者向交易添加见证，每个见证包含调用脚本和验证脚本（确定性代理脚本）。

阶段四：链上验证

主合约验证确定性代理见证，计算 Neo 见证数量，从 EVM 签名中恢复以太坊地址，检查组合签名载荷是否达到统一操作阈值。

3.2 核心验证方法

3.2.1 Verify 方法：两阶段验证模型

`Verify(accountId)` 方法是由确定性代理脚本调用的统一验证入口点，实现了区分交易验证和应用执行的两阶段验证模型。

两阶段验证架构

阶段一：验证触发器（交易验证阶段）

- **时机**：在交易进入内存池时发生，早于应用执行。
- **限制**：此阶段无法验证 EVM 签名，因为 EIP-712 签名数据位于交易参数中，在纯验证阶段不可访问。
- **行为**：检查交易结构是否匹配标准 AA 封装调用，然后仅计算 Neo `CheckWitness` 签名。

阶段二：应用触发器（执行阶段）

- **时机**：当目标合约在执行阶段显式调用 `Runtime.CheckWitness(accountAddress)` 时发生。
- **行为**：`VerifyContext` 确保只有当前预授权的目标合约可以通过，从而解锁 EVM 类型化验证逻辑。

3.2.2 CheckNativeSignatures：纯 Neo 签名验证

`CheckNativeSignatures` 方法使用 `Runtime.CheckWitness()` 处理纯 Neo 签名验证，用于验证阶段和纯 Neo 执行路径。

纯 Neo 原生验证

使用场景

在初始验证阶段（交易校验）调用，以高效拒绝未经身份验证的载荷；在纯原生操作的应用执行阶段调用，无需元数据开销。

机制与限制

直接在节点层利用 `Runtime.CheckWitness`。由于仅针对加载在 `Witnesses` 数组中的签名证据，它在数学上无法评估编码到执行参数中的 EVM 类型化数据签名。

3.3 CheckMixedSignatures：统一阈值算法

混合授权协议的核心创新是 `CheckMixedSignatures` 算法，它将 Neo 见证验证与 EVM 签名验证统一为单一阈值检查。

算法定义：

```
CheckMixedSignatures(roles, threshold, verifiedSigners):
  输入: roles           - 授权地址列表 (UInt160[])
        threshold       - 所需最小签名数 (int)
        verifiedSigners - 从签名中恢复的 EVM 地址 (UInt160[])
  输出: 布尔值, 表示是否满足阈值

  count <- 0
```

```
for each address in roles:
  matched <- false
  // 步骤一：检查是否为 EVM 签名者
  if verifiedSigners != null:
    for each signer in verifiedSigners:
      if address == signer:
        count <- count + 1
        matched <- true; break
  // 步骤二：检查是否为 Neo 签名者
  if not matched:
    if Runtime.CheckWitness(address):
      count <- count + 1
  if count >= threshold: return true
return false
```

统一计数机制

密码学隔离

EVM 签名使用 secp256k1 和 Keccak256 在合约应用上下文中独立验证；Neo 见证利用节点的原生 secp256r1 核心机制。两条验证路径不能互相拦截、覆盖或伪造。

公平贡献

无论来源如何，每个映射到已知账户角色的密码学验证密钥，对统一阈值贡献恰好一份授权权重。

顺序无关性

计数算法遍历链上存储的规范授权角色数组，不受载荷插入顺序影响。

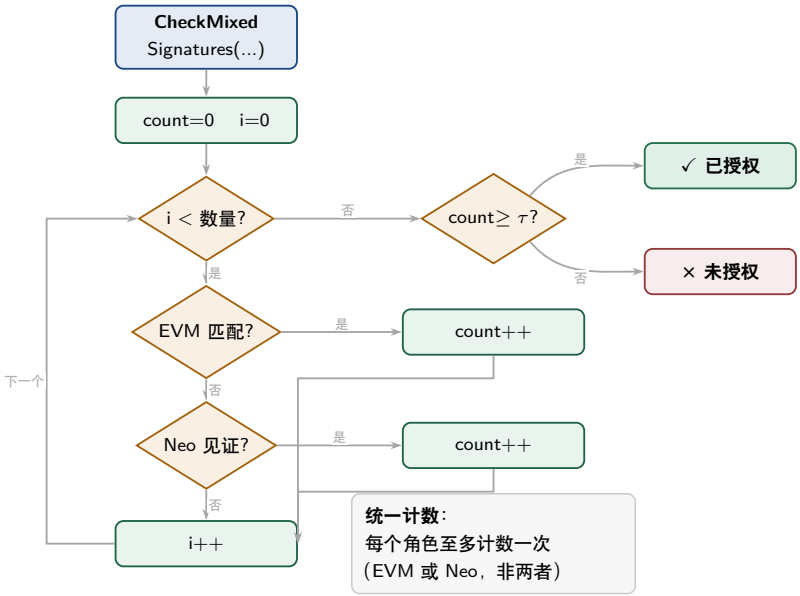

图 4: CheckMixedSignatures 中的统一阈值评估：恢复的 EVM 签名者与 Neo 见证合并计入一个策略阈值。

3.4 EIP-712 签名验证详情

EIP-712 验证过程确保 EVM 签名与特定操作进行密码学绑定，无法被重放或滥用。

域分隔符构造：

$$\begin{aligned}\text{domainSep} = \text{keccak256}(&\text{EIP712_DOMAIN_TYPEHASH}\\ &\|\text{keccak256}(\text{“Neo N3 Abstract Account”})\\ &\|\text{keccak256}(\text{“1”})\\ &\|\text{uint256}(\text{chainId})\\ &\|\text{address}(\text{verifyingContract}))\end{aligned}$$

其中 chainId = 894710606（Neo N3 测试网），verifyingContract 为主合约哈希。

3.5 状态管理与随机数机制

混合授权协议维护一个账户范围的元交易随机数。虽然公共 getter 签名仍接受签名者参数以保持向后兼容，但存储的随机数以逻辑账户而非个别 EVM 签名者为键。这与当前实现相匹配，使重放保护整体附着于账户执行流。

对于每个抽象账户，主合约存储：

- **元交易随机数**：账户随机数存储键下的单个单调递增值。
- **最后活跃时间戳**：每当账户成功满足授权路径时更新。
- **临时执行上下文**：仅在活跃执行路径期间使用的范围验证上下文和已恢复签名者上下文。

处理 EVM 元交易时，主合约：

1. 在使用基于地址的封装器时，从提供的 AA 地址解析逻辑账户。
2. 重新计算该账户和目标调用的预期 `argsHash` 和 EIP-712 摘要。
3. 读取账户范围的存储随机数，并要求提供的随机数精确匹配。
4. 验证 secp256k1 签名，派生恢复的签名者地址，然后在分发前递增存储的账户随机数。

3.6 EIP-712 类型化数据结构

图 5: 混合授权工作流：EVM 类型化数据签名与 Neo 执行合并，在一个阈值下完成授权。

类型化数据的精确结构如下：

```
EIP712Domain {
  name: "Neo␣N3␣Abstract␣Account"
  version: "1"
```

```
  chainId: <Runtime.GetNetwork()>
  verifyingContract: <masterContractHash>
}

MetaTransaction {
  accountAddress: address
  targetContract: address
  methodHash: bytes32
  argsHash: bytes32
  nonce: uint256
  deadline: uint256
}
```

两点重要细节：其一，签名身份为 AA 地址而非原始 `accountId`。其二，方法字段为方法名的 Keccak-256 哈希而非原始字符串，保证链上重建紧凑且确定。

`argsHash` 通过 Neo 的 `StdLib.Serialize` 序列化方法参数后对序列化字节进行 Keccak-256 哈希计算得出。验证成功后账户范围的随机数递增：$n \leftarrow n + 1$。截止时间标准化为 Neo 的毫秒制 `Runtime.Time`，当 Runtime.Time $>$ $\text{deadline}_{\text{normalized}}$ 时拒绝元交易。

3.7 验证算法

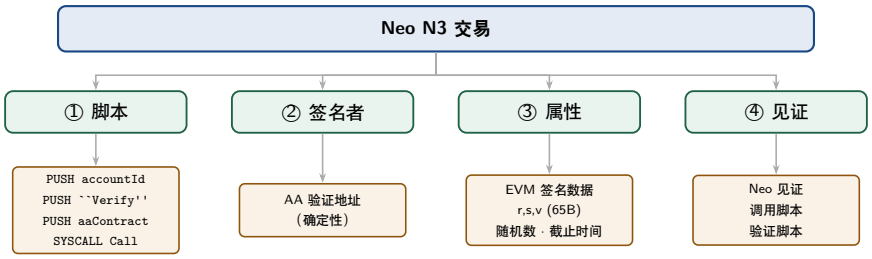

图 6: AA 执行的 Neo N3 交易布局，突出显示验证代理签名者、可选 EVM 签名载荷及 Neo 见证。

高层阈值算法

1. 初始化 count $= 0$。

2. 对每个 Neo 见证：若派生地址在管理员/管理者集合中，count 递增。

3. 若存在 EVM 签名：恢复公钥，派生以太坊地址，若该地址在管理员/管理者集合中，count 递增。

4. 当且仅当 count $\geq$ threshold 时接受交易。

4 安全分析

4.1 威胁模型

假设敌手 $\mathcal{A}$ 控制网络消息层，可观察所有内存池交易并提交任意载荷，但在计算上无法：

- 破解离散对数假设；

- 在没有私钥的情况下伪造 ECDSA 签名（secp256k1 或 secp256r1 曲线）；
- 在 SHA-256 或 Keccak-256 中找到哈希碰撞。

4.2 形式安全定义

定义 4.1 (签名不可伪造性游戏)**.** *敌手 $\mathcal{A}$ 获胜条件：*

1. $\mathcal{A}$ 接收目标公钥 $\{pk_1, \ldots, pk_n\}$；

2. $\mathcal{A}$ 进行自适应签名查询 $q_1, \ldots, q_m$；

3. $\mathcal{A}$ 输出消息和签名集合 $(m^*, \{\sigma_i^*\})$，且未对 m^* 进行过查询；

4. 验证接受 $(m^*, \{\sigma_i^*\})$，但有效签名数量少于阈值 τ。

优势定义为：$Adv_{\mathcal{A}} = \Pr[\mathcal{A}$ 获胜$]$。

安全属性	执行机制
确定性	对于固定输入状态（accountId、签名、操作），执行结果在所有节点上严格可预测。由 Neo N3 VM 确定性执行和共享合约路由执行。
不可伪造性	敌手无法伪造授权载荷。由 secp256r1（Neo 见证）和 secp256k1（EVM ecrecover）上的离散对数假设执行。
重放抵御	截获的签名无法被重新使用。随机数机制防止跨时间重放。Neo 见证绑定到唯一交易哈希；EVM 签名绑定到账户范围的序列随机数。
可靠性	包含少于预定义 τ 个有效签名的授权被严格拒绝，在 `CheckMixedSignatures` 过程中自动检查。
前向安全性	被盗签名者密钥无法重写过去的历史事件。严格单调递增的随机数明确使任何向回签名的尝试无效。

表 1: Neo-AA 账户抽象协议的主要安全属性摘要。

定理 4.1 (阈值正确性)**.** *在标准 ECDSA 抗碰撞假设下，主合约的授权引擎当且仅当有效且唯一的签名者数量 $n \geq \tau$ 时接受操作。*

证明概要. `CheckMixedSignatures` 过程通过评估集合 $\mathcal{R}_A \cup \mathcal{R}_M$ 将签名者归一化到单一域。对于 Neo 见证，`CheckWitness` 确保见证包含有效签名。对于 EVM 签名，合约派生 $H(m)$，ecrecover 对每个签名元组 (r, s, v) 确认恰好一个以太坊地址。由于映射步骤防止重复，合约精确累积经验证成员的数量。 □

定理 4.2 (重放界)**.** *针对状态 St_i 构造的截获 EVM 签名 $\sigma = (r, s, v)$ 无法在状态 St_j $(j > i)$ 被重新应用以构造有效评估。*

证明概要. EIP-712 MetaTransaction 载荷将 σ 绑定到账户随机数 n_i。在 St_i 中的执行块成功验证后，合约严格执行 $n_{i+1} = n_i + 1$。任何后续评估 St_j $(j > i)$ 将要求 $n_j \geq n_{i+1}$。由于 $n_i \neq n_j$，σ 的签名验证将在新鲜类型化数据哈希上失败。 □

5 相关工作

5.1 账户抽象系统

ERC-4337。以太坊的 ERC-4337 架构通过用户操作、打包器和入口点合约将用户与外部账户解耦。它是可编程钱包行为的重要参考，但假设以太坊原生执行模型，通常依赖按账户的钱包合约或账户特定验证逻辑。相比之下，Neo-AA 将授权集中在共享 Neo 合约中，专为异构 Neo/EVM 签名者集合设计。

Argent 与 Safe。 Argent 和 Safe 等生产级智能钱包系统展示了监护人恢复、角色分离、阈值策略和模块化账户控制的价值。它们与 Neo-AA 的关联是架构层面的：它们为恢复和治理界面提供参考，但它们仍是单生态系统设计，未解决在一个阈值规则中同时计算 Neo 见证和 EVM 类型化签名的问题。

5.2 阈值授权与跨生态系统授权

阈值签名文献。 GG20 和 CGGMP 风格的阈值 ECDSA 系统在一条曲线和一种签名模型下提供分布式签名，降低密钥管理风险，但不直接解决一条链使用见证验证而另一条链使用恢复类型化数据签名的异构验证环境问题。

桥接与互操作系统。大多数跨链系统专注于消息传递或资产转移，旨在证明一条链上的事件使另一条链能够响应。Neo-AA 的目标是授权互操作性：一个账户策略应接受来自不同钱包生态系统的签名者，而无需强制这些生态系统共享签名格式或桥接信任假设。

5.3 Neo-AA 的定位

相对于上述系统，Neo-AA 占据独特的设计点：Neo N3 上的零按账户部署、确定性地址生成、混合签名阈值计数，以及无需更改主要执行接口即可扩展的恢复层。其新颖性不在于任何单一原语，而在于将这些原语组合成以地址为优先、面向生产的账户模型的方式。

6 讨论

6.1 局限性

当前系统局限性

曲线覆盖

当前实现支持 Neo 的原生 secp256r1 见证路径和 EVM 风格的 secp256k1 类型化数据验证。支持其他生态系统（如 Ed25519）需要新的签名验证例程和新的序列化或域分隔规则。

串行化元交易排序

当前账户范围的随机数简单且安全，但也意味着一个账户的元交易在一个共享的重放保护序列中推进。这是有意识的安全导向选择。

执行成本

EVM 公钥恢复和签名验证相比纯 Neo 见证验证增加了计算开销。该设计对于预期的治理和账户操作工作流实际可行，但并非完全免费。

6.2 设计权衡

架构权衡

共享合约 vs. 按账户部署

共享合约最小化部署成本并简化通用逻辑升级，但将更多复杂性推向确定性路由、范围执行上下文和细致的内部状态分离。

地址优先可用性 vs. 逻辑标识符存储

将 AA 地址作为主要公开身份暴露改善了用户体验和前端交互，而在内部保留 `accountId` 使存储、索引和验证器集成保持简洁。规范将标识符视为互补关系。

统一验证 vs. 分离执行平面

在一个阈值路径中计算 Neo 见证和 EVM 恢复签名者产生简单且富有表达力的策略模型，但需要两阶段验证循环和对临时脚本上下文的细心处理。

6.3 运营架构

模块化合约结构

部署的主合约遵循刻意模块化的拆分以保持可维护性：

- `AccountLifecycle`：账户创建、验证和确定性地址绑定。
- `StorageAndContext`：规范存储键、地址映射、活跃度跟踪、验证上下文和执行锁。
- `ExecutionAndPermissions`：原生执行、混合授权、策略执行和合约分发。
- `MetaTx`：EIP-712 摘要构造、参数哈希、随机数管理和已恢复签名者处理。
- **管理模块**：角色变更、Dome 配置、预言机回调和可选外部验证器。

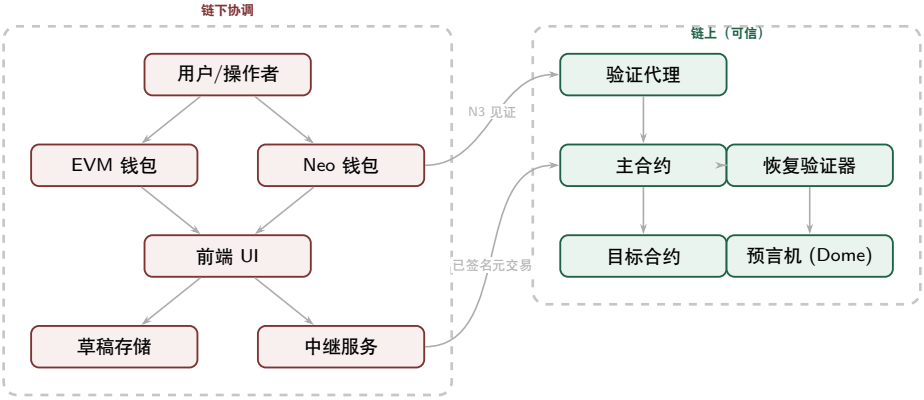

图 7: Neo-AA 的信任边界视图：链下协调负责准备交易和签名；链上合约是唯一的授权权威。

6.4 未来方向

有前景的扩展方向包括：更广泛的生态系统支持（如附加签名曲线和域格式）、基于零知识技术的隐私保护授权、更丰富的批量执行模式，以及对完整执行上下文不变量的机器检验验证。另一个自然方向是加强以域为中心的发现，使 `.matrix` 命名在保持从属于链上授权状态的同时，更深入地参与用户旅程。

6.5 部署详情

面向生产的实现已针对 Neo N3 测试网工作流进行验证，并附带可复现的合约构件、集成脚本和协作交易准备的前端支持。部署模型假设：一个主合约、确定性 AA 地址派生、基于地址的执行入口点，以及与核心合约一起编译的

恢复验证器构件。

7 结论

本文介绍了 Neo-AA——一种 Neo N3 账户抽象架构，在一个共享主合约中统一了基于原生见证的授权与 EVM EIP-712 类型化签名。核心贡献不仅是签名验证，而是一个完整的账户模型，将确定性 AA 寻址、地址优先用户交互、混合阈值授权和可扩展恢复结合在单一链上控制平面下。

规范明确了两个与实现对齐的要点：其一，外部有意义的身份是确定性 AA 地址，而 `accountId` 仍是用于存储、确定性路由和验证器兼容性的内部键；其二，混合授权路径以账户为中心：恢复的 EVM 签名者和 Neo 见证归一化到一个权限引擎，重放保护通过账户范围的随机数而非签名者特定计数器推进。

正是这些选择使 Neo-AA 既实用又概念清晰。一个主合约可以支持多个账户而无需按账户部署；用户可以通过 AA 地址或 `.matrix` 名称发现和操作账户；恢复逻辑可以演进而无需重新设计执行界面。未来工作包括更广泛的生态系统支持、隐私保护授权，以及对支撑系统安全性的执行上下文不变量的更正式验证。